

How Agent Foundry Works

Arghya Mukherjee

September 12, 2026

CONTENTS

How Agent Foundry Works	
1. The one-sentence version	
2. The mental model: an employee, not a chatbot	
3. The whole architecture, in one picture	
4. The loop every agent runs: think → act → critique	
5. Two engines run that same loop — pick one per agent, or per call	
6. When one agent isn't enough: the six topologies	
7. Why these architectural choices — justified, not just described	
8. Module-by-module reference — what every file actually does	
Foundation — the types everything else plugs into	
The loop itself — Layer 02	
Runtime, tools & model — Layers 03–05	
Context Layer — Layer 06	
Guardrails & security	
Eval, observability & optimization	
Integration & cross-cutting	
9. How you'd actually build something with this	
Four entry points, in increasing order of governance	
The foundry CLI — operating an agent from the shell	
10. The four reference example apps	
11. What's actually proven — the test suite	
12. CI/CD and deployment	
What runs on every push and PR (.github/workflows/)	
Deployment	
13. Security posture — the OWASP LLM Top 10, honestly cross-checked	
14. Backup & disaster recovery	
15. Native vs. LangGraph — measured, not claimed	
16. Being honest about what's not finished	
17. Worked example: a Pricing & Promotions Agent, end to end	
The scenario	

How Agent Foundry Works

Mapping the scenario onto every layer

Building it (the governed path, §9's option 4)

Walking the turn through the loop (§4)

Where this would grow next

18. Where to go next

How Agent Foundry Works

A plain-language, full-coverage walkthrough of the whole repository — read this before `ARCHITECTURE.md` (the formal spec) or the `README` (the dense, fully-cited version). This document trades some precision for clarity; where it simplifies, the linked source file has the exact behavior.

1. The one-sentence version

Agent Foundry is a **library**, not a hosted service: you write plain Python (or a YAML file), and it gives you an `Agent` object that can think, call tools, check itself before answering, and ask a human when it's unsure — with permissions, spending limits, and an audit trail built in from the start, not bolted on later.

Everything else in this document is one of three things:

- **the loop** an agent runs every turn (think → act → check itself),
- **the scaffolding** around that loop (who's allowed to do what, which tools exist, where memory lives, how you know it's working), and
- **the proof** that it actually works (546 tests across 65 files, CI, a benchmark suite, an OWASP threat-model cross-check).

2. The mental model: an employee, not a chatbot

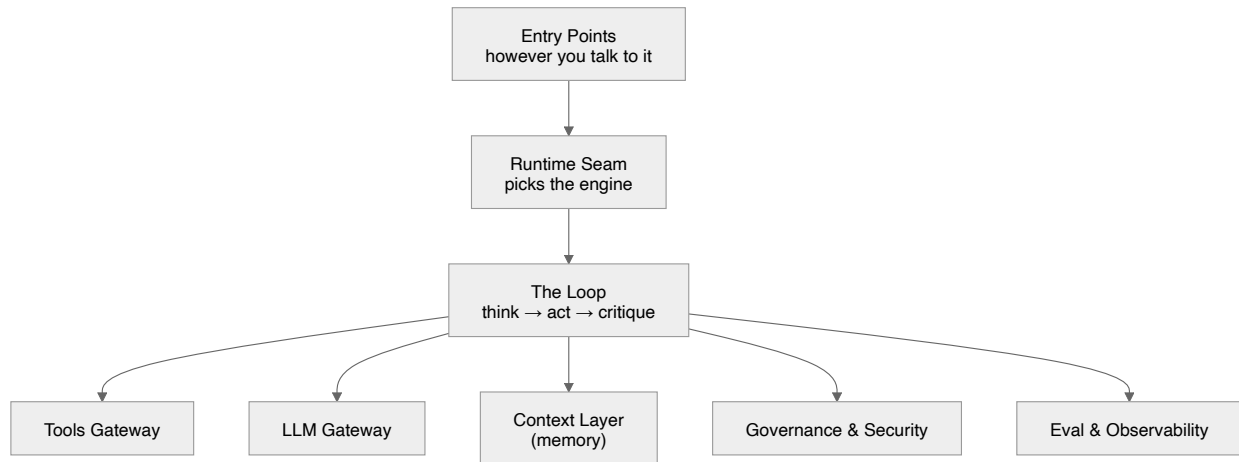
The easiest way to hold this codebase in your head is to stop thinking “chatbot” and think “**new employee with a job description.**”

Real-world concept	Agent Foundry equivalent	File
The employee's job description	<code>instructions</code> (a system prompt)	<code>prompts.py</code>
What they're allowed to touch	<code>Policy</code> (allowed tools, spend cap, approval list)	<code>contracts.py</code>
Their badge / who they are	<code>Identity</code> (id, tenant, roles)	<code>contracts.py</code>
The tools on their desk	<code>ToolRegistry</code> (functions, MCP servers, REST APIs, SQL)	<code>tools_gateway.py</code>
Their notebook / CRM history	<code>MemoryStore</code> (working, episodic, semantic/RAG, profiles)	<code>context.py</code>
A manager who reviews risky decisions	Guardrails + human-in-the-loop approval	<code>guardrails.py</code> , <code>escalation.py</code>
A manager who checks their work before it goes out	The critique step	<code>orchestration.py</code> / <code>core/native_engine.py</code>
HR's record of everything that happened	<code>AuditLog</code>	<code>security.py</code>
The org chart, if you hire more than one employee	<code>Workflow</code> topologies (supervisor, swarm, debate, ...)	<code>core/native_orchestration.py</code> , <code>orchestration.py</code>

You build **one** `AgentConfig` (the job description + permissions + tools + memory), and that same config can be run as a solo employee, or wired into a team. Nothing about the employee changes when you put them on a team — only how turns get routed to them.

3. The whole architecture, in one picture

Zooming out before zooming in — the simplest version of the whole system, eight boxes, one flow:



Every box is a real layer, not a diagram-only concept. Layer by layer, what each one is and why it exists as its own thing rather than folded into another:

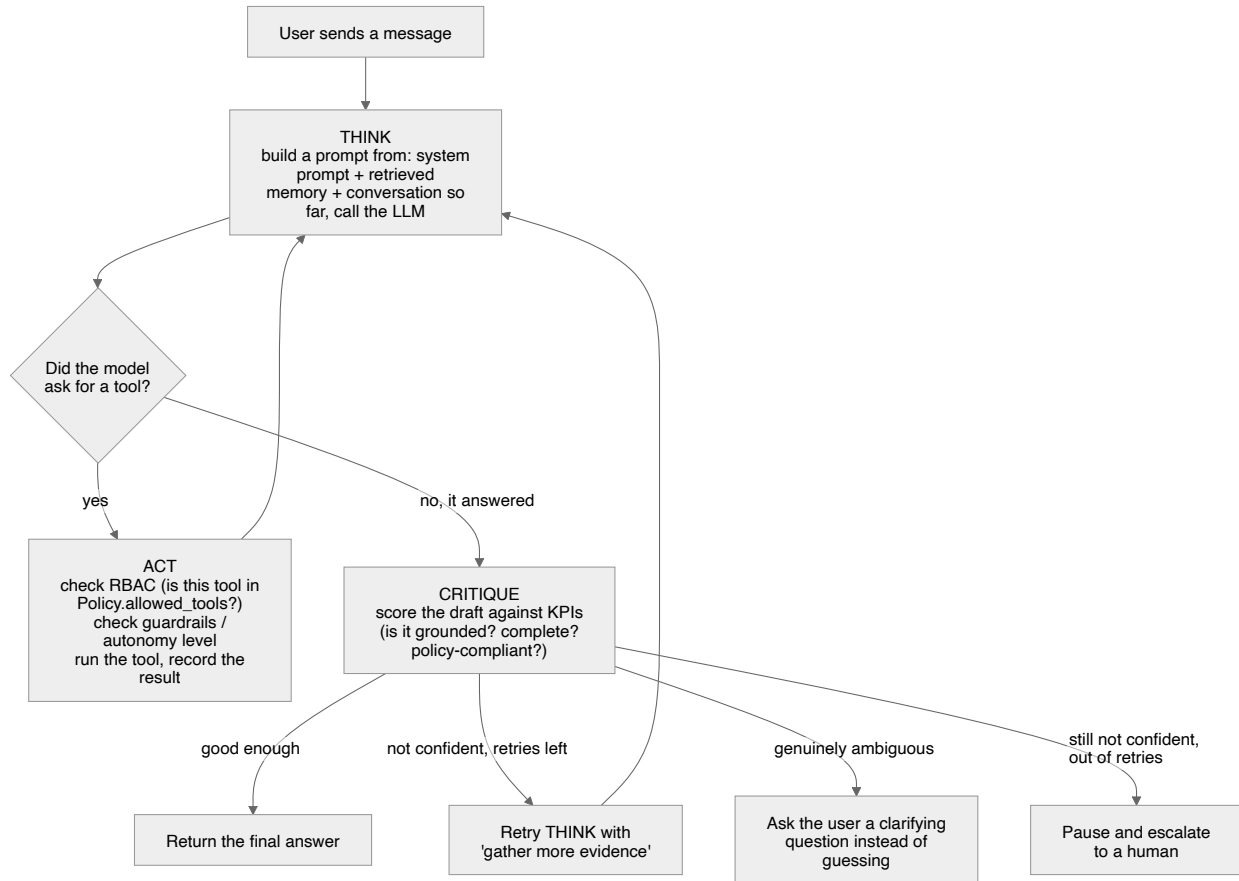
Layer	Purpose	Key modules	Why it's a separate layer
Entry Points	However a request reaches an agent	<code>Agent / Workflow</code> (Python), <code>AgentSpec</code> (YAML/JSON), <code>cli.py</code> (<code>foundry</code> shell command), <code>serve.py</code> (HTTP chat API), <code>channels.py</code> (Slack), <code>a2a_bridge.py</code> (agent-to-agent)	Six different front doors, all building the <i>same</i> <code>AgentConfig</code> — a Slack message and a direct Python call get identical governance, not a lesser version
Runtime Seam	Picks which engine actually executes the loop	<code>core/protocols.py</code> 's <code>WorkflowEngine</code> , <code>core/engines.py</code> 's <code>RUNTIMES</code> registry	One keyword switch (<code>runtime=</code>) instead of a rewrite — a new engine (Temporal, say) registers here without <code>Agent</code> itself changing
The Loop	The actual agent behavior — this is the “intelligence”	<code>orchestration.py</code> (LangGraph path), <code>core/native_engine.py</code> (native path)	Every topology and every runtime is built from the same three primitives: think, act, critique — one loop to understand, two engines to run it
Tools Gateway	Everything an agent can <i>do</i>	<code>tools_gateway.py</code> , <code>mcp_tools.py</code> , <code>http_tools.py</code> , <code>crewai_bridge.py</code> , <code>autogen_bridge.py</code> , <code>data_connectors.py</code>	RBAC, caching, rate limiting, and idempotency happen in one place, regardless of whether the tool is a local function, an MCP server, or someone else's agent
LLM Gateway	Everything an agent <i>thinks with</i>	<code>llm_gateway.py</code>	One contract (<code>complete(messages, model=...)</code>) — no other layer imports a vendor SDK directly, so swapping or failing over models doesn't ripple upward
Context Layer	Everything an agent <i>remembers</i>	<code>context.py</code>	Working, episodic, semantic (RAG), and cross-session profile memory behind one API — kept separate from the loop so memory can be upgraded (in-memory → Chroma) without touching orchestration
Governance & Security	Who's allowed to do what, and the proof of it	<code>contracts.py</code> , <code>guardrails.py</code> , <code>security.py</code> , <code>escalation.py</code> , <code>policy_engine.py</code>	RBAC, guardrails, signed tool manifests, and an audit trail run on <i>every</i> call through the loop — load-bearing, not a wrapper that can be skipped
Eval & Observability	How you know it's actually working	<code>kpi.py</code> , <code>core/evalgate.py</code> , <code>observability.py</code> , <code>experiments.py</code>	Watches every step from the side rather than sitting in the request path — and can fail a release (§12), not just log one after the fact

A request comes in through one of the six front doors, the runtime seam picks the engine, and the loop runs — touching tools, the model, and memory as it needs them, checked by governance at every action, watched by eval/observability the whole time.

4. The loop every agent runs: think → act → critique

This is the actual engine. Every single-agent turn — regardless of which “runtime” executes it (more on that in §5) — is this loop:

How Agent Foundry Works



Walking it through with a concrete example (this is literally what `examples/support_agent.py` does):

1. A customer asks *"What's the status of order A100?"*
2. **Think**: the agent's system prompt + any relevant memory (nothing needed here) go to the LLM. The model decides it needs the `lookup_order` tool.
3. **Act**: before running `lookup_order`, a Policy Decision Point checks more than a simple allowlist — is `lookup_order` in this identity's `Policy.allowed_tools`? Is it flagged `requires_approval`? Does it push spend-so-far over the thread's cost cap? Does it reach an allowed host (`egress_hosts`)? (All clear.) It runs, returns `"A100: shipped, eta 2026-09-05"`. If the model asked for several tools in the same turn, the cleared ones run *concurrently*, not one at a time — the loop only serializes around an approval pause, so nothing ordered before it starts before a human sees the prompt.
4. Back to **Think**: the model now has the tool result and writes a real answer.
5. **Critique**: the answer is scored (did it actually answer the question? is it concise?). It passes, and is returned.

Now the *risky* path — *"Refund \$20 for order A100"*:

1. **Think** decides to call `issue_refund`.
2. **Act** checks policy — `issue_refund` is in `requires_approval`. Instead of running it, the loop **pauses** and surfaces an approval request.
3. A human (in the CLI, the web chat UI, or a Slack thread) approves or denies it.
4. Approved → the loop **resumes** exactly where it left off, actually runs the refund, and continues to Critique → final answer. Denied → the model is told and answers accordingly.

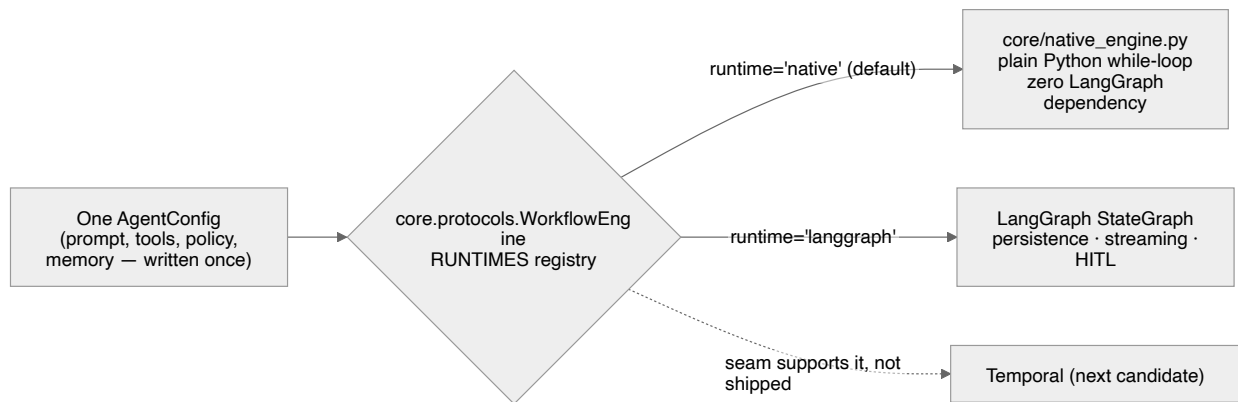
That pause/resume mechanic is the same one used for “I’m not sure, let me ask a person” — a destructive tool call and a genuine escalation both go through the same interrupt point, just for different reasons (`Policy.requires_approval` vs. the critique step’s own escalate branch).

5. Two engines run that same loop — pick one per agent, or per call

This is the single most important architectural decision in the codebase, so it’s worth being very explicit about it.

The loop above is described once, conceptually. It is then *implemented twice*:

- `runtime="native"` (the default) — `core/native_engine.py`. A plain Python `while` loop. No LangGraph. No `StateGraph`. No `interrupt()`. Just dataclasses and functions. `pip install agent-foundry` with zero extras can run this.
- `runtime="langgraph"` (opt-in, `pip install agent-foundry[langgraph]`) — the exact same steps, but executed as nodes in a LangGraph `StateGraph`, which gives you LangGraph’s own persistence/checkpointing and streaming machinery for free.



Why does this matter, practically?

- **You write your agent once.** Same prompt, same tools, same `Policy`. You choose the engine with one keyword, per-agent or per-call (`agent.run(msg, runtime="langgraph")` overrides the default for a single call).
- **It’s proven interchangeable, not just parallel.** `tests/test_native_engine.py` runs the *exact same test scenarios* as `tests/test_core_agent.py` (the LangGraph path), just against the other engine — same RBAC, same budgets, same critique behavior.
- **Native is faster and lighter** — see §15 for the measured numbers.
- **LangGraph still earns its keep** for two specific things native doesn’t do yet: durable checkpointing via LangGraph’s own storage mechanism, and pausing mid-conversation inside a blackboard/debate hand-off (a documented, tested gap — see §16).

All **six multi-agent topologies** (§6) also run on either engine — it’s not “native = single agent only.”

6. When one agent isn’t enough: the six topologies

`Agent` covers one employee. `Workflow` is the org chart for when you need more than one — and every shape below is built from the *same* three primitives (`think`, `act`, a router), just wired differently.

Topology	Real-world analogy	What actually happens
Supervisor	A dispatcher routes each call to the right specialist	One LLM reads the request and picks which named agent handles it (fails <i>closed</i> — an unrecognized route retries once, then either goes to a configured <code>fallback_agent</code> or raises, rather than silently guessing)
Swarm	Peer hand-off, no manager	Any specialist can transfer the conversation directly to another peer — no central router
Blackboard	A shared whiteboard in a war room	Several agents read and write to one shared workspace (facts, hypotheses, evidence, open questions) instead of talking to each other directly
Debate	A panel + a judge	N agents answer independently; a judge agent reads all answers and synthesizes a final one
Fanout	Assign the same task to 50 people at once	One agent applied to many inputs <i>concurrently</i> (bounded — <code>max_concurrency</code> , default 32 — so you don't spin up a thread per item)
DAG	An assembly line	A fixed pipeline of deterministic steps with dependencies — for when you need reliability, not judgement, at every hop

Every one of these is available as `Workflow.supervisor(...)`, `Workflow.swarm(...)`, etc., and every one takes the same `runtime="native"` or `runtime="langgraph"` keyword as a single `Agent`.

Two things worth knowing if you're picking a topology for something real:

- **Human-in-the-loop across a topology hop** — pausing mid-turn for approval — is solid for supervisor and swarm on both engines. For blackboard/debate it's solid on native (`core/native_orchestration.py`'s `_PausableTurns`), but a confirmed, tested gap on LangGraph (an interrupt inside one participant's turn gets silently lost). This is called out directly in the README rather than glossed over.
- Native multi-agent *topology* state (who's turn it is, a paused approval, debate round number) is **not** yet wired to a durable `StateStore`. A single agent's own conversation state *is* durable if you pass `state_store=` (in-memory, Postgres, or Redis). If the process dies mid-topology-turn, you lose that topology's position; a single agent's state survives.

7. Why these architectural choices — justified, not just described

Every major structural decision in this codebase was a trade-off, not a default. This table names each one, what was actually chosen, why, and what it costs — so the architecture reads as a set of decisions with reasons, not just a list of components.

Decision	What was chosen	Why (the justification)	Trade-off / cost
How extension points are defined	A Python <code>Protocol</code> (structural typing) for every swappable piece — <code>WorkflowEngine</code> , <code>Tool</code> , <code>Memory</code> , <code>StateStore</code> , <code>GuardrailChecks</code> , <code>Evaluator</code> , <code>PolicyEngine</code> , <code>VectorStore</code> , and more	A replacement doesn't need to subclass anything, only match the shape — and every protocol here has at least two real implementations proven , not just declared possible (Anthropic + OpenAI; regex guardrails + LLM guardrails; OPA + Cedar; in-memory + Chroma)	Less compile-time enforcement than an abstract base class — conformance is structural and tested, not declared
How many execution engines exist, and which is default	Two — <code>native</code> (plain Python, default) and <code>Langgraph</code> (opt-in) — behind one <code>WorkflowEngine</code> seam	<code>pip install agent-foundry</code> needs zero <code>LangGraph</code> install to run an agent; <code>native</code> is measurably faster and lighter (§15); <code>LangGraph</code> stays available for teams that specifically need its own durable checkpointing or multi-agent topology-hop HITL	Two implementations of the same loop to keep behaviorally identical — mitigated by running the <i>same</i> test scenarios against both (§11), not by inspection alone
How much autonomy an agent has	A graduated six-level scale (<code>L0</code> — <code>L5</code>), not a binary approve/deny	Real-world risk isn't binary — “may draft, not act” and “may act on anything policy allows” are genuinely different postures or organizations actually operate on a spectrum of delegated trust	More states to reason about than a boolean — but it maps to how a real approvals process already works, rather than forcing everything through one gate
How a tool call is authorized	A deterministic Policy Decision Point (allowlist, spend cap, egress hosts, requires-approval) — not “trust the model to follow its instructions”	A model can be talked out of its own instructions (prompt injection); a check enforced entirely outside the model's control can't be	Someone has to maintain the policy explicitly — it can't infer intent, only enforce what's declared
What guardrails run by default	Regex/heuristic pattern matching, with <code>LLMGuardrails</code> (LLM-judgment) available as an opt-in upgrade	A zero-dependency, zero-latency, zero-extra-cost baseline that works with no API key and no extra model call on every turn	Regex has real recall limits — stated plainly in the docs (§13) rather than oversold, precisely so a team knows to layer <code>LLMGuardrails</code> on top for high-stakes input
What happens when a budget is exceeded	Fails closed — raises and stops the turn — rather than silently capping and continuing	A runaway loop should stop visibly, not keep going in some degraded, unnoticed state	A legitimate long-running task can hit the ceiling and need its limit deliberately raised — that friction is the point, not a bug
Where evaluation sits in the lifecycle	A release gate (<code>core/evalgate.py</code> 's <code>.passes(thresholds)</code>), callable from <code>foundry.eval</code> in CI), not only a post-hoc dashboard	Catches a prompt or topology regression <i>before</i> it ships, the same way a test suite catches a code regression	Needs a maintained, versioned eval dataset per agent — ongoing work, not a one-time setup cost
How other agent	Interop bridges (MCP, A2A, CrewAI, AutoGen) — wrap their agent as one	A team already invested in CrewAI or AutoGen shouldn't	The integration is only as deep as “callable as a tool” — not a merge

Decision	What was chosen	Why (the justification)	Trade-off / cost
frameworks are treated	<code>ToolSpec</code> , or hand one of ours to theirs	have to rewrite into a new DSL just to use this framework's governance	of two frameworks' own state/memory models
How an agent can be configured	Both a Python API (<code>Agent(...)</code>) and a declarative <code>AgentSpec</code> (YAML/JSON) — same governed path either way	Lets a non-engineer change a prompt, a tool wiring, or a policy value without touching Python, while an engineer keeps full code-level control when needed	Anything needing a live Python object (a custom <code>KPI</code> , a hand-built <code>LLMGateway</code>) still needs the Python path — YAML covers the structurally-expressible common case, not everything
How the framework stays cloud-portable	No module in <code>agent_foundry/</code> imports a cloud-specific SDK; the container is the only portability boundary	The same image runs unmodified on AWS, GCP, Azure, any Kubernetes cluster, or a bare VM	A managed cloud service (a managed vector DB, a managed queue) has to be wired in at the deployment layer, by design — not something the framework abstracts away for you
How “pause for a human” is implemented	One interrupt/resume mechanism, shared by both “this tool needs approval” and “the critique step isn't confident”	Fewer moving parts to get right — a single pause/resume path serving two different triggers, instead of two mechanisms that could quietly drift apart	A caller resuming a paused turn needs to know which case it is to respond sensibly — both go through the same <code>resume(approved=...)</code> shape
How tool calls in one turn are dispatched	Concurrently by default, serializing only around an approval pause	Throughput — a turn calling three independent tools shouldn't pay for them one at a time	The per-turn spend-so-far check is computed once at the top of the turn rather than re-read after each call — a deliberate, documented accuracy/latency trade, not an oversight

8. Module-by-module reference — what every file actually does

This is the answer to “what does each piece of this codebase do, and why would I ever touch it.” Every module in `agent_foundry/` is one row, grouped by layer. Read it top to bottom once and you have the whole system; come back to it as a lookup table afterward.

FOUNDATION — THE TYPES EVERYTHING ELSE PLUGS INTO

Module	What it does	Why it matters
<code>contracts.py</code>	Defines <code>Identity</code> , <code>Policy</code> , <code>AutonomyLevel</code> (L0–L5), <code>ToolSpec</code> , <code>ToolResult</code> , <code>ToolCall</code> , <code>LLMResponse</code> , <code>GuardrailResult</code> — plain dataclasses, no logic	Read this file first — every other layer is built to plug into these exact shapes
<code>prompts.py</code>	Loads a system prompt from a plain <code>.md</code> or <code>.txt</code> file instead of a Python string literal	Keeps prompts editable by a non-engineer, versionable in git like any other content file
<code>agent_spec.py</code>	<code>AgentSpec.from_yaml(...)</code> — the declarative alternative to writing Python: a YAML/JSON file describing tools, policy, and a named critique evaluator	Lets a team hand a non-Python-writing operator a config file instead of a script; <code>foundry run --spec</code> and <code>foundry eval</code> both consume it

THE LOOP ITSELF — LAYER 02

Module	What it does	Why it matters
<code>orchestration.py</code>	<code>make_think_node</code> , <code>make_act_node</code> , <code>make_critique_node</code> — the three primitives — plus all 7 <code>build_*_graph</code> topology builders (LangGraph path)	Everything else in the repo is a slot this file's nodes call into
<code>core/native_engine.py</code>	<code>NativeEngine</code> — a second, framework-free implementation of the identical think/act/critique loop	This is what <code>runtime="native"</code> actually runs — no LangGraph import anywhere in it
<code>core/native_orchestration.py</code>	A native counterpart of each multi-agent topology — pausable/resumable, bounded-concurrency, real event streaming	Same framework-free idea as <code>native_engine.py</code> , one level up (multi-agent instead of single-agent)
<code>core/engines.py</code>	<code>RUNTIMES = {"native": ..., "langgraph": ...}</code> — the registry <code>Agent.__init__</code> looks a runtime name up in	The actual extension point: add a <code>TemporalWorkflowEngine</code> , register it here, done — <code>Agent</code> never needs to change
<code>core/protocols.py</code>	The <code>Protocol</code> contracts every pluggable piece satisfies: <code>WorkflowEngine</code> , <code>Tool</code> , <code>Memory</code> , <code>StateStore</code>	Structural typing, not inheritance — a replacement doesn't subclass anything, it just matches the shape
<code>core/agent.py</code>	<code>Agent</code> and <code>Workflow</code> — the actual public classes you import	The thing that hides “build a graph, invoke it with a dict, inspect an interrupt” behind <code>.run()</code> , <code>.stream()</code> , <code>.resume()</code>
<code>core/execution_context.py</code>	<code>ExecutionContext</code> — carries the run id, thread id, session id, user id, tenant id, permissions, and budget for one call	The one object you pass to <code>.run(msg, context=...)</code> to identify who's calling and which conversation this is
<code>core/result.py</code>	<code>RunResult</code> , <code>result_from_graph_output()</code>	Normalizes native and LangGraph's differently-shaped raw output into one consistent return type
<code>core/run.py</code>	<code>Run</code> , <code>RunStatus</code> — eight lifecycle states, from <code>STARTED</code> and <code>RUNNING</code> through <code>WAITING_HUMAN</code> and <code>WAITING_EVENT</code> to a terminal <code>COMPLETED</code> , <code>FAILED</code> , or <code>CANCELLED</code>	<code>Agent.start()</code> 's formal lifecycle — pause, unpause, cancel, retry, fork, replay, and wait-for-event, as methods on <code>Run</code>
<code>core/registries.py</code>	<code>PromptRegistry</code> , <code>PolicyRegistry</code> , <code>EvalRegistry</code> — named lookup tables	Lets <code>Agent(instructions="support_v2", policy="strict")</code> resolve a name instead of importing an object directly
<code>core/state_store.py</code>	<code>MemoryStateStore</code> , <code>PostgresStateStore</code> — both implement <code>StateStore</code> 's load/save/delete contract	What makes a single native agent's conversation survive a process restart — pass <code>state_store=</code>

Module	What it does	Why it matters
<code>core/model_router.py</code>	Capability-based model selection — <code>ModelRequest</code> → <code>routes[task]</code>	Picks cheap vs. hard model by declared task complexity, not a hardcoded model name
<code>core/tool_decorator.py</code>	<code>@tool(...)</code> — a decorator that turns a plain function into a <code>ToolSpec</code> with real timeout/cache_ttl/permissions metadata	The ergonomic way to register a tool without hand-building a <code>ToolSpec</code>
<code>core/evalgate.py</code>	<code>run_eval()</code> , <code>EvalCase</code> , <code>Scorecard</code> , <code>.passes(thresholds)</code>	Turns evaluation into a release gate — fail a build if an agent regresses against a versioned dataset

RUNTIME, TOOLS & MODEL — LAYERS 03–05

Module	What it does	Why it matters
<code>runtime.py</code>	<code>RunBudget</code> , <code>LatencyBudget</code> , <code>CircuitBreaker</code> , <code>RateLimiter</code> , <code>SLATracker</code> , each behind a swappable <code>*Like</code> Protocol	Per-thread cost/step/latency ceilings, retries, and circuit breaking — fails <i>closed</i> , not silently capped
<code>distributed.py</code>	<code>RedisRunBudget</code> , <code>RedisRateLimiter</code> , <code>RedisToolCache</code> , <code>RedisSLATracker</code> , <code>RedisCostLedger</code> , <code>RedisStateStore</code>	The real, cross-replica version of everything in <code>runtime.py</code> — what makes a multi-instance deployment actually share state
<code>tools_gateway.py</code>	<code>ToolRegistry</code> , <code>ToolCache</code> , <code>InMemoryIdempotencyStore</code> , <code>tool_json_schema</code>	Every tool call funnels through here: RBAC check, result caching, rate limiting, idempotency
<code>mcp_tools.py</code>	<code>MCPToolSource</code> — connects to any MCP server (stdio or HTTP) and registers its tools into the same registry	Any MCP-compatible tool server becomes indistinguishable from a local Python function
<code>http_tools.py</code>	<code>http_tool()</code> — wraps any REST endpoint as a <code>ToolSpec</code>	No MCP server needed for the common case of “call this API”
<code>autogen_bridge.py</code>	<code>autogen_as_tool()</code> — wraps a Microsoft AutoGen agent as one tool call	Interop, not a rewrite — use an existing AutoGen agent from inside a governed Agent Foundry agent
<code>crewai_bridge.py</code>	<code>crewai_as_tool()</code> — wraps a CrewAI crew as one tool call	Same idea, for CrewAI — and the reverse direction works too (hand one of <i>your</i> tools to <i>their</i> agent)
<code>data_connectors.py</code>	<code>DataSource</code> protocol, <code>SQLiteDataSource</code> , <code>data_query_tool()</code> — SELECT-only, table-allowlisted	Structured data (SQL/warehouses), distinct from <code>context.py</code> 's unstructured RAG
<code>llm_gateway.py</code>	<code>LLMGateway</code> , <code>AnthropicProvider</code> , <code>OpenAIProvider</code> , <code>MultiProvider</code> , <code>PromptCache</code> , <code>ModelRegistry</code>	One contract (<code>complete(messages, model=...) → LLMResponse</code>) every provider satisfies — nothing else in the framework imports a vendor SDK directly

CONTEXT LAYER — LAYER 06

Module	What it does	Why it matters
<code>context.py</code>	<code>VectorStore</code> protocol (<code>InMemoryVectorStore</code> , <code>ChromaVectorStore</code>), <code>KnowledgeGraphStore</code> , <code>ProceduralMemory</code> , <code>MemoryStore</code> (working/episodic/semantic/profiles), <code>ContextEngine</code>	Everything an agent remembers — within a thread, across threads, and across sessions for the same user

GUARDRAILS & SECURITY

Module	What it does	Why it matters
<code>guardrails.py</code>	<code>GuardrailEngine</code> , <code>LLMGuardrails</code> , <code>redact()</code> , <code>looks_like_injection()</code>	Input/output/action gates — regex/heuristic by default, an LLM-judgment option layered on top
<code>security.py</code>	<code>ToolManifestRegistry</code> , <code>EgressPolicy</code> , <code>CredentialVault</code> , <code>VaultCredentialProvider</code> , <code>AuditLog</code> , <code>EncryptedJSONLAuditLog</code>	Signed tool manifests (catches schema drift), egress allowlisting, real secrets management, an audit trail
<code>policy_engine.py</code>	<code>OPAPolicyEngine</code> (real Rego via a running OPA server), <code>CedarPolicyEngine</code> (AWS Cedar, in-process)	Real policy-as-code, as an alternative or addition to the built-in <code>Policy</code> dataclass
<code>escalation.py</code>	<code>EscalationTicket</code> , <code>QueueEscalator</code>	The third outcome besides auto-approve/deny — hand a case to a human queue instead of blocking outright
<code>sandbox.py</code>	<code>run_sandboxed()</code> , <code>code_execution_tool()</code>	Restricted-builtins, wall-clock-timeout code execution — process-level isolation, not a container

EVAL, OBSERVABILITY & OPTIMIZATION

Module	What it does	Why it matters
<code>kpi.py</code>	<code>KPI</code> , <code>KPIBoard</code> , <code>KPIResult</code> + ten reference scoring functions (efficiency, groundedness, policy adherence, cost, ...)	Composable scoring the critique step (and eval gate) score against — weight them however a use case needs
<code>eval.py</code>	<code>EvalHarness</code> , <code>JSONLEvalSink</code>	Atomic / component / flow / overall evaluation levels, in-memory or durable
<code>eval_dataset.py</code>	<code>EvalDataset</code> , <code>EvalCase</code> — a versioned JSON format for eval cases	What <code>foundry eval</code> and <code>core/evalgate.py</code> actually score an agent against
<code>observability.py</code>	<code>Tracer</code> , <code>OTelTracer</code> , <code>Metrics</code> , <code>CostLedger</code> , <code>check_alerts()</code>	Tracing (in-memory or real OpenTelemetry), a cost ledger that closes the moment a task finishes, threshold alerting
<code>benchmark.py</code>	<code>BenchmarkCase</code> , <code>CaseResult</code> , <code>BenchmarkReport</code> , <code>run_benchmark()</code>	A regression suite against any compiled graph — catches a prompt/topology change quietly making things worse
<code>experiments.py</code>	<code>Experiment</code> , <code>ExperimentTracker</code>	Deterministic A/B variant assignment (stable per identity) + per-variant metric aggregation
<code>feature_flags.py</code>	<code>FeatureFlagProvider</code> protocol, <code>StaticFeatureFlagProvider</code>	On/off and percentage-rollout switches, bucketed by a stable hash so an identity's answer never flaps
<code>reinforcement.py</code>	<code>PromptOptimizer</code> , <code>PreferenceStore</code>	Closes eval signal back into prompts (curates best-scoring exemplars) and into export-ready chosen/rejected pairs for DPO/distillation
<code>planning.py</code>	<code>Objective</code> , <code>Planner</code> , <code>StrategySelector</code> , <code>BanditSelector</code>	Scores candidates (a model, a topology) against a weighted <code>KPIBoard</code> ; <code>BanditSelector</code> learns from observed rewards instead of a fixed formula

INTEGRATION & CROSS-CUTTING

Module	What it does	Why it matters
<code>events.py</code>	<code>EventBus</code> protocol, <code>InMemoryEventBus</code> , <code>KafkaEventBus</code> , <code>wire_event_driven()</code>	Pub/sub — <code>Agent.on("order.delayed")</code> makes an agent react automatically to an external event
<code>blackboard.py</code>	<code>Blackboard</code> , <code>parse_post()</code>	The shared reasoning workspace the blackboard topology's agents read/write to
<code>a2a_bridge.py</code>	<code>agent_card_for()</code> , <code>build_a2a_app()</code>	Makes an agent discoverable and callable over the open Agent2Agent protocol
<code>serve.py</code>	<code>build_http_app()</code> , <code>invoke_graph_chat_turn()</code>	Wraps any compiled graph as a minimal FastAPI app: <code>POST /chat</code> , <code>GET /health</code> , plus a browser chat UI with inline HITL approval
<code>channels.py</code>	<code>build_slack_app()</code> , <code>verify_slack_signature()</code>	A real, tested Slack Events API integration (HMAC signature check, URL-verification handshake) — the same pattern extends to SMS/email/Teams
<code>versioning.py</code>	<code>VersionStore</code> protocol, <code>FileVersionStore</code>	Immutable-version + current-pointer rollback for prompts/policy documents
<code>i18n.py</code>	<code>LocaleSpec</code> , <code>register_locale()</code> , <code>format_currency()</code> , <code>format_date()</code>	Locale-aware prompt variants and response formatting
<code>batch.py</code>	<code>Scheduler</code> protocol, <code>IntervalScheduler</code> , <code>run_batch()</code> , <code>BatchReport</code>	Runs a compiled graph once per item, concurrently, for offline/bulk jobs — distinct from <code>build_fanout_graph</code> 's in-turn parallelism
<code>scaffold.py</code>	<code>create_agent()</code> — the <code>python -m agent_foundry.scaffold</code> CLI	Generates a runnable agent file + prompt file with everything wired except the prompt content and tool bodies (see §9)
<code>quickstart.py</code>	<code>plug_and_play_agent()</code> , <code>to_langchain_tool()</code>	The minimal, real-LangChain entry point for teams who want speed over governance first (see §9)
<code>cli.py</code>	The <code>foundry</code> command: <code>init</code> , <code>run</code> , <code>eval</code> , <code>serve</code> , <code>inspect</code> , <code>trace</code>	The operational surface — <code>scaffold</code> , <code>run</code> , <code>score</code> , <code>serve</code> , and <code>introspect</code> an agent from the shell, not just from Python
<code>ui/console.py</code>	A CLI REPL that drives any compiled graph, including inline HITL approval prompts	The reference implementation of “how do I actually talk to this thing” outside a web UI

9. How you'd actually build something with this

FOUR ENTRY POINTS, IN INCREASING ORDER OF GOVERNANCE

You can start on the left and grow into the right without rewriting your tools (`quickstart.to_langchain_tool()` bridges a tool already registered in a governed `ToolRegistry` back into the simple path).

1. **Scaffold it** — fastest way to a runnable file:

```
python -m agent_foundry.scaffold sales_agent --tools lookup_lead,send_email
```

Writes `prompts/sales_agent.md` and `agents/sales_agent.py` — a runnable script with gateways, guardrails, eval, runtime, and tracer already wired. The only TODOs left: the prompt's content and each tool function's body.

2. **quickstart.plugin_and_play_agent(...)** — a few lines, real LangChain tool-calling, no `ToolSpec` /JSON schema — plain Python functions with a docstring become the tool schema automatically. For when speed matters more than RBAC/budgets, at first.
3. **orchestration.build_agent_graph(...)** — the fully governed path, spelled out by hand: identity/policy, tools, model/budget/guardrails, memory, compile. What `scaffold.py`'s generated file and `examples/support_agent.py` both build on.
4. **Agent(name, instructions, tools=[...])** — the same governed path as #3, but you never see `LangGraph`, never build an invoke dict by hand, never inspect an interrupt object:

```
from agent_foundry import Agent, ExecutionContext

def lookup_order(order_id: str) -> str:
    """Look up an order by id."""
    return db.get(order_id)

agent = Agent("support_agent", "You are a support agent.", tools=[lookup_order])
result = agent.run("status of order A100?", context=ExecutionContext(thread_id="t1"))
print(result.content)
```

THE `FOUNDRY` CLI — OPERATING AN AGENT FROM THE SHELL

`cli.py` gives you six commands, so a whole agent lifecycle doesn't require writing a driver script every time:

Command	What it does
<code>foundry init <name></code>	Scaffolds a new agent (thin wrapper over <code>scaffold.py</code>)
<code>foundry run <script></code> (or <code>--spec agent.yaml</code>)	Runs a scaffolded agent script, or a declarative <code>AgentSpec</code>
<code>foundry eval <script> <dataset.json> --thresholds '{...}'</code>	Scores an <code>Agent</code> against a versioned dataset and exits non-zero if it fails the thresholds — this is <code>core/evalgate.py</code> 's release gate, callable from CI
<code>foundry serve <script></code>	Serves a script's top-level <code>app</code> or <code>graph</code> over HTTP
<code>foundry inspect</code>	Shows the installed version and which optional extras (<code>[anthropic]</code> , <code>[langgraph]</code> , <code>[redis]</code> , ...) are actually available
<code>foundry trace <file.jsonl></code>	Tails a JSONL eval/audit log file

10. The four reference example apps

Each demonstrates a different slice of the framework, each has both an imperative `agent.py` and a declarative `agent.yaml`, and each (except one, see below) ships a versioned `eval_dataset.json`:

Example	What it's for	What it specifically proves
<code>examples/support_agent.py</code>	A support bot: order lookup, refunds, customer profiles	The whole stack in one file — RBAC, RAG-grounded answers, a destructive tool pausing for approval, cross-session profile memory, A/B prompt testing, the audit trail and dashboard, all exercised end to end
<code>examples/research_agent/</code>	A RAG-only agent that answers strictly from a small seeded knowledge base, citing sources	<code>citation_correctness_kpi</code> (does every <code>[doc:N]</code> marker name a real source?) and <code>composite_grounding_kpi</code> (is the wording actually supported by what was retrieved?)
<code>examples/commerce_agent/</code>	A shopping assistant: search/recommend (read-only) + place an order (destructive)	The same HITL pattern as <code>support_agent</code> 's refund, plus trajectory evaluation (<code>expected_tool_sequence</code> , <code>must_request_approval</code>) — was the <i>path</i> correct, not just the final answer
<code>examples/autonomous_workflow/</code>	An order-monitor driven through <code>core.run.Run</code> 's formal lifecycle, reacting to events, self-verifying its own answers	<code>Agent.start()</code> → <code>Run</code> (STARTED/RUNNING/WAITING_HUMAN/WAITING_EVENT/...), <code>Agent.on()</code> event wiring, and critique escalating to a human only when its own evidence doesn't support its answer — not on every action

Why `autonomous_workflow` has no `Workflow.dag/.supervisor` flourish: `core.evalgate.run_eval` — and the whole eval-as-release-gate story — is built against a plain `Agent.run(...)`, not the different `run(items)` / `run(inputs)` shapes `Workflow.fanout/.dag` expose. A multi-agent demo would have been flashier but couldn't be scored the same consistent way the other three are.

11. What's actually proven — the test suite

546 test functions across 65 files (`tests/`), plus 3 more test files inside each example app. This isn't a "we have tests" checkbox — a few categories are worth knowing about because they prove specific architectural claims, not just "the code runs":

What's being proven	How
Native and LangGraph are truly interchangeable	<code>tests/test_native_engine.py</code> / <code>tests/test_native_orchestration.py</code> run the identical scenarios <code>tests/test_core_agent.py</code> / <code>tests/test_orchestration.py</code> run against LangGraph — same fixtures, same assertions, different engine
Concurrency guarantees are real, not assumed	<code>tests/test_native_engine_concurrency.py</code> , <code>tests/test_native_orchestration_concurrency.py</code> , <code>tests/test_orchestration_concurrent_tools.py</code> — forced interleaving via a blocking test provider (deterministic, not a timing-based flaky test): two turns on the <i>same</i> thread never run concurrently, different threads do
Streaming parity between engines	<code>tests/test_agent_stream.py</code> asserts native and LangGraph yield the <i>same chunk count</i> for the same scripted scenario, not just “at least one chunk”
Durable state actually survives a restart	<code>tests/test_native_engine_state_store.py</code> — a brand-new <code>NativeEngine</code> sharing only the <code>StateStore</code> continues an earlier one's thread, including a mid-turn approval pause
State store backends work against the real thing	<code>tests/test_state_store_backends.py</code> runs the full contract against a live reachable Redis/Postgres, skipping cleanly when neither is up — not mocked
Topology-level human-in-the-loop	<code>tests/test_native_orchestration_hitl.py</code> proves pause/resume across a specialist's tool-approval interrupt for all four applicable topologies
The declared LangGraph HITL gap is real, not hypothetical	<code>tests/test_orchestration.py</code> has a test reproducing it directly, named <code>test_debate_silently_loses_a_debaters_tool_approval_interrupt_KNOWN_GAP</code>
Every protocol has ≥ 2 real, swappable implementations	<code>tests/test_protocol_substitutability.py</code> , <code>tests/test_workflow_engine_protocol.py</code>
The public API surface doesn't silently change	<code>tests/test_public_api_surface.py</code>
Distributed primitives actually share state cross-replica	<code>tests/test_agent_distributed_redis.py</code> , <code>tests/test_distributed.py</code> — against a live Redis

12. CI/CD and deployment

WHAT RUNS ON EVERY PUSH AND PR (`.GITHUB/WORKFLOWS/`)

Workflow	What it checks
<code>test.yml</code>	lint (<code>ruff check</code> , pinned version), typecheck (<code>mypy</code> , pinned, against real installed stubs), pytest on Python 3.10/3.11/3.12 with every optional extra installed (<code>[all, test]</code>), coverage gated at 80%+ (measured baseline is ~84% — a real margin, not a threshold tuned to just barely pass)
<code>build.yml</code>	Builds the sdist + wheel, installs it fresh, verifies <code>foundry --help</code> and <code>import agent_foundry</code> both work from the built artifact — not just from an editable install
<code>security.yml</code>	<code>pip-audit</code> against every installed dependency (<code>[all]</code> extras), plus a weekly scheduled run (Monday 06:00 UTC) so a newly-disclosed CVE gets caught even in a week with no code changes. Known, unfixed upstream CVEs are excluded explicitly by ID with a comment explaining why — not silently ignored

DEPLOYMENT

`serve.py` wraps any compiled graph as a FastAPI app; the repo's `Dockerfile` packages it into one container (`python:3.12-slim` , health-checked on `/health`). No module in `agent_foundry/` imports a cloud-specific SDK, so the same image runs unmodified on AWS ECS/Fargate/App Runner, GCP Cloud Run, Azure Container Apps, any Kubernetes cluster, or a bare VM — the container is the entire portability boundary. `distributed.py`'s Redis-backed `budget/cache/rate-limiter/SLA/cost-ledger` classes are what make that container safe to run as more than one replica (see §8).

13. Security posture — the OWASP LLM Top 10, honestly cross-checked

`docs/OWASP_LLM_TOP10.md` walks through all ten risks against specific, already-tested code — not aspirational coverage. The condensed version:

Risk	Mitigated by	Status
LLM01 Prompt Injection	<code>guardrails.py</code> regex markers + optional <code>LLMGuardrails</code>	Direct mitigation, not a trained classifier
LLM02 Sensitive Info Disclosure	<code>redact()</code> and <code>check_output</code> PII patterns; secrets never touch prompts (<code>VaultCredentialProvider</code>)	Direct mitigation
LLM03 Supply Chain	<code>ToolManifestRegistry</code> detects tool signature drift	Partial — no SBOM/dependency scanning built in (that's <code>pip-audit</code> in CI, not runtime code)
LLM04 Data/Model Poisoning	RAG passages carry source metadata for reviewer visibility	Partial — no automated ingestion trust check
LLM05 Improper Output Handling	<code>check_output</code> , JSON-schema validation, <code>sandbox.py</code> 's restricted execution	Direct mitigation
LLM06 Excessive Agency	<code>AutonomyLevel</code> (L0–L5), <code>Policy.allowed_tools</code> , <code>requires_approval</code> + <code>interrupt</code> , <code>RunBudget</code> and <code>LatencyBudget</code> , <code>escalation.py</code>	Direct mitigation — the risk the framework is most deliberately built around
LLM07 System Prompt Leakage	Injection markers catch “reveal your system prompt” attempts	Direct mitigation, with a named gap (no output-side echo check)
LLM08 Vector/Embedding Weaknesses	<code>ChromaVectorStore</code> verified thread-isolated	Partial — no embedding-poisoning detection (open research problem)
LLM09 Misinformation	<code>db_match_kpi</code> (real fact-check), <code>reference_check_kpi</code> , <code>llm_judge_kpi</code>	Direct mitigation, at eval time
LLM10 Unbounded Consumption	<code>RunBudget</code> , <code>LatencyBudget</code> , <code>RateLimiter</code> , <code>CircuitBreaker</code> , <code>max_steps_per_thread</code>	Direct mitigation

Bottom line: 7 of 10 have direct, tested mitigations; the other 3 have partial coverage with an explicit, named gap — stated plainly rather than stretched to look complete.

14. Backup & disaster recovery

There's no framework *code* for this — it's a deployment decision about which data lives where, documented as a runbook in `docs/BACKUP_DR.md`. The short version:

State	Durable by default?
Conversation/thread state (LangGraph checkpoints, or native without a <code>state_store=</code>)	No — in-process only, gone on restart, unless you pass a real <code>checkpointer/ StateStore</code>
Semantic memory (RAG documents/embeddings)	Depends on the <code>VectorStore</code> — <code>InMemoryVectorStore</code> is not durable; <code>ChromaVectorStore(path=...)</code> persists to disk
Audit log	Durable (append-only file) as long as the volume it lives on is — use <code>EncryptedJSONLAuditLog</code> for anything sensitive
Cost/eval/SLA history (in-process trackers)	No — this is observability, not source-of-truth state; snapshot it yourself if you need history to survive a restart

Making conversation state durable is one constructor argument: pass a real `checkpointers=` (LangGraph path) or `state_store=` (native path — `MemoryStateStore`, `PostgresStateStore`, or `distributed.py`'s `RedisStateStore`) instead of accepting the in-memory default. Whatever you actually persist to (a Postgres instance, a Redis instance, a Chroma path, a JSONL audit file) then needs the same backup discipline as any other stateful service — the framework doesn't do this for you, deliberately, since it depends entirely on the surrounding deployment.

15. Native vs. LangGraph — measured, not claimed

`benchmarks/native_vs_langgraph.py` runs identical scripted scenarios (single-agent, single-agent with a tool call, and all 6 multi-agent topologies) through both runtimes, using an in-process scripted LLM provider (no network) — so this measures the two runtimes' *own* overhead, not live-model latency:

Topology	Runtime	P50 (ms)	P95 (ms)	Memory (KB)
agent	native	0.07	0.11	49.1
agent	langgraph	1.79	3.12	2317.9
supervisor	native	0.08	0.10	20.4
supervisor	langgraph	2.29	2.78	559.6
blackboard	native	0.37	0.42	151.5
blackboard	langgraph	64.07	69.41	1170.6
debate	native	0.22	0.28	168.3
debate	langgraph	47.63	52.86	916.5

(Full table with fanout/DAG/tool-calling variants and cost/eval columns in the README.) Cost is identical between runtimes for the same scenario — the runtime doesn't change how many model calls a topology makes, only how it executes the loop around them. The gap that matters is latency and memory, and it's most dramatic on LLM-call-heavy sequential topologies (blackboard, debate), where LangGraph's per-node graph-execution overhead compounds across several turns. Run it yourself: `python benchmarks/native_vs_langgraph.py`.

16. Being honest about what's *not* finished

The project's own README is unusually blunt about this, and it's worth carrying that same honesty forward rather than presenting a rosier picture:

- **Default guardrails are regex/heuristic**, not a trained classifier. `LLMGuardrails` exists as a stronger option, but it's opt-in, not the default.
- **Native multi-agent topology state is process-local** (see §6) — a crash mid-topology-turn loses that turn's position, even though a single agent's own state survives via `state_store=`.
- **Blackboard/debate human-in-the-loop on LangGraph** silently swallows an interrupt inside a participant's turn — a confirmed, reproduced bug (`tests/test_orchestration.py`'s `test_debate_silentlyloses_a_debaters_tool_approval_interrupt_KNOWN_GAP`), not a hypothetical.
- **Temporal** is a designed-for, not-yet-built third runtime — the `WorkflowEngine` seam supports adding it (implement the protocol, register it in `RUNTIMES`, done), but nobody's written `TemporalWorkflowEngine` yet.
- **3 of 10 OWASP LLM risks** (supply chain, data poisoning, embedding weaknesses) have partial, not complete, coverage (§13).

- **Backup/DR is a runbook, not code** (§14) — the framework makes durability *possible* (swap in a real checkpointer/StateStore/VectorStore path) but doesn't back anything up for you.

17. Worked example: a Pricing & Promotions Agent, end to end

None of the four shipped reference apps (§10) are a pricing agent — this is a new use case, built from the same primitives, to show how a team would actually reach for each layer of the architecture for a real business problem.

THE SCENARIO

A retail company's merchandising team wants an internal assistant that can:

1. Answer "what's the current price / active promotion for this SKU?" — read-only.
2. **Apply** a promo code to a live order — this changes money changing hands, so it's destructive.
3. **Recommend** (not apply) a discount for a customer segment — a draft, never executed directly.
4. Never invent a discount that isn't real, and never approve one that violates a minimum margin.
5. Escalate to a human pricing manager when a requested discount is unusually large, instead of just refusing.

That's a realistic mix of read-only lookups, a destructive action, a policy-grounded judgment call, and a human escalation path — which is exactly what the architecture in §3 was built for.

MAPPING THE SCENARIO ONTO EVERY LAYER

Layer (from §3)	What it looks like for this agent
Entry Points	A Python <code>Agent</code> for now; later, <code>channels.py</code> puts it in the merchandising team's Slack, and <code>agent_spec.py</code> lets a pricing analyst (not an engineer) edit the prompt and policy via YAML
Tools Gateway	<code>get_price(sku)</code> , <code>get_active_promotions(sku)</code> — read-only; <code>apply_promo_code(order_id, code)</code> — <code>destructive=True</code> ; <code>recommend_discount(segment, sku)</code> — non-destructive, returns a suggestion only, touches no real state
LLM Gateway	<code>task="default"</code> for most turns; could route a high-stakes discount decision to <code>task="hard"</code> via <code>core/model_router.py</code> if the cheaper default model's judgment isn't trusted enough for large discounts
Context Layer	Semantic memory (RAG) holds the actual Promotions Policy document, so the agent cites real rules instead of guessing; <code>data_connectors.py</code> reaches the product catalog's real price/margin table (structured data, not RAG — the two are deliberately separate, see §3's Context Layer row)
Governance & Security	<code>Policy.requires_approval = {"apply_promo_code"};</code> <code>Policy.allowed_tools</code> scoped so a customer-support-facing version of this agent can <code>recommend_discount</code> but never <code>apply_promo_code</code> — same tool code, different <code>Policy</code> , no rewrite
Eval & Observability	A custom <code>margin_safety</code> KPI flags any recommended or applied discount that would push margin below a floor; <code>core/evalgate.py</code> scores the agent against a dataset of "customer asks for an unearned discount" cases before every prompt change ships

BUILDING IT (THE GOVERNED PATH, §9'S OPTION 4)

```

from agent_foundry import Agent, ExecutionContext
from agent_foundry.contracts import Policy, ToolSpec
from agent_foundry.context import MemoryStore
from agent_foundry.kpi import KPI
from agent_foundry.orchestration import CritiqueConfig

# --- Tools Gateway: real functions, one flagged destructive ---
def get_price(sku: str) -> str:
    return f"{sku}: $49.99"

def get_active_promotions(sku: str) -> str:
    return f"{sku}: SAVE20 (20% off, expires 2026-09-30)"

def apply_promo_code(order_id: str, code: str) -> str:
    return f"applied {code} to order {order_id}"

def recommend_discount(segment: str, sku: str) -> str:
    return f"suggest 10% off {sku} for segment {segment} (within margin floor)"

tools = [
    ToolSpec("get_price", "Look up the current price for a SKU", {"sku": "string"}, get_price),
    ToolSpec("get_active_promotions", "List active promotions for a SKU", {"sku": "string"},
get_active_promotions),
    ToolSpec("apply_promo_code", "Apply a promo code to a live order", {"order_id": "string", "code":
"string"},
        apply_promo_code, destructive=True),
    ToolSpec("recommend_discount", "Suggest (do not apply) a discount for a segment", {"segment":
"string", "sku": "string"},
        recommend_discount),
]

# --- Governance: who can do what ---
policy = Policy(
    allowed_tools=frozenset({t.name for t in tools}),
    requires_approval=frozenset({"apply_promo_code"}),
    max_cost_usd_per_thread=1.0,
    max_steps_per_thread=10,
)

# --- Context: ground the agent in the real policy doc, scoped to this thread ---
thread_id = "pricing-desk-1"
memory = MemoryStore()
memory.semantic.upsert(
    thread_id,
    "Discounts over 30% require VP approval. Minimum margin floor is 15% on all SKUs.",
    {"source": "promotions_policy.md"},
)

# --- Eval: a domain-specific KPI, checked by the critique gate every turn ---
def _margin_safety_score(ctx: dict) -> float:
    return 0.0 if "50%" in ctx["output_text"] or "60%" in ctx["output_text"] else 1.0 # simplified:
flag unrealistic discounts

def _critique_context(state, draft: str) -> dict:
    return {"output_text": draft}

critique = CritiqueConfig(
    kpi=KPI(name="margin_safety", score=_margin_safety_score, threshold=0.5),
    context=_critique_context,
    escalate_threshold=0.1, # only a genuinely unsafe discount pauses for a human; most turns just
flag

```

```

)

agent = Agent(
    "pricing_promo_agent",
    "You are a pricing and promotions assistant for the merchandising team. "
    "Only apply a promo code that is actually active. Never approve a discount "
    "that violates the margin floor in policy – recommend, don't guess.",
    tools=tools,
    policy=policy,
    memory=memory,
    critique=critique,
)

result = agent.run("Apply code SAVE20 to order 0-991", context=ExecutionContext(thread_id=thread_id))

```

WALKING THE TURN THROUGH THE LOOP (§4)

1. **Think:** the model reads the prompt + the RAG-retrieved policy passage, decides to call `apply_promo_code`.
2. **Act:** the Policy Decision Point sees `apply_promo_code` is `destructive=True` and in `requires_approval` — the turn **pauses** instead of moving money. A pricing manager approves it in the same Slack thread `channels.py` wires up.
3. Back to **Think** → **Critique:** the `margin_safety` KPI scores the outcome; if a future request asked for an unrealistic 60% discount, it would score `0.0` — below both `threshold` and `escalate_threshold` — and the turn would pause for a human reviewer rather than silently applying it.
4. Everything — the approval decision, the applied code, the KPI score — lands in the `AuditLog` (§8's `security.py`), because this is a real financial action, not a chat reply.

WHERE THIS WOULD GROW NEXT

- **Split into a topology** (§6) once the domain grows: a `Workflow.supervisor` with a “Pricing” specialist (price/margin lookups) and a “Promotions” specialist (code logic and campaign rules), routed by intent — same tools, same `Policy` objects, no rewrite.
- **Raise the autonomy level** (`contracts.AutonomyLevel`, §8) from `L3_APPROVAL` to `L4_POLICY_BOUND` for small, pre-approved discount bands (under 10%, say), while keeping `L3` for anything larger — a policy change, not a code change.
- **Add `foundry eval`** (§9's CLI) to a CI job so a prompt tweak that starts recommending unsafe discounts fails the build before a merchandiser ever sees it.

18. Where to go next

If you want...	Read
The exhaustive, grep-verified reference (every module, every claim cited)	<code>README.md</code>
The formal layered spec this document simplifies	<code>docs/ARCHITECTURE.md</code>
A step-by-step build guide	<code>docs/IMPLEMENTATION_GUIDE.md</code>
The full OWASP LLM Top 10 walkthrough	<code>docs/OWASP_LLM_TOP10.md</code>
The backup/DR runbook	<code>docs/BACKUP_DR.md</code>
A working example end to end	<code>examples/support_agent.py</code> (single agent, tools, RAG, HITL refund, A/B test, dashboard — all in one file)
The other three reference apps	<code>examples/research_agent/</code> , <code>examples/commerce_agent/</code> , <code>examples/autonomous_workflow/</code>